

E Commerce Product Delivery Prediction

Executed project report with saved notebook outputs, tables, and charts

Project Description

E Commerce Product Delivery Prediction

Project Summary

This project is a clean, student-friendly implementation of a delivery performance classification. The main goal is to predict whether an e-commerce order will reach on time. The notebook keeps the workflow simple: load the dataset, understand the columns, clean the data, train a baseline model, and check how well the model performs.

No external repository links or profile links are included in this description. Everything needed to understand and run the project is kept inside this folder.

Problem Statement

A raw dataset is useful only when it is converted into a clear decision-making workflow. In this project, the data is processed step by step so that important patterns can be found and a machine learning model can be trained. The expected output is based on Reached.on.TimeY.N.

Files Included

File	Purpose	Quick Note
ECommerce.csv	Main data source for this folder	2000 sampled rows x 12 columns
Notebook file	Complete Python workflow	Includes loading, cleaning, training, and evaluation
PDF report	Human-readable project summary	Matches the updated project structure

Important Columns

Column	Role
ID	Input feature used in the modelling workflow
Warehouseblock	Input feature used in the modelling workflow
ModeofShipment	Input feature used in the modelling workflow
Customercarecalls	Input feature used in the modelling workflow
Customerrating	Input feature used in the modelling workflow
CostoftheProduct	Input feature used in the modelling workflow
Priorpurchases	Input feature used in the modelling workflow
Productimportance	Input feature used in the modelling workflow
Gender	Input feature used in the modelling workflow
Discountoffered	Input feature used in the modelling workflow
Weightingms	Input feature used in the modelling workflow
Reached.on.TimeY.N	Target / output column used in the modelling workflow

Workflow Followed

1. Load the data from the local dataset file present in the same project folder.
2. Inspect the structure using shape, column names, missing values, and basic statistics.
3. Clean the dataset by removing empty columns, handling missing values, and keeping only useful features.
4. Prepare the model input by separating features and the target column.
5. Train a baseline model using a simple scikit-learn pipeline with preprocessing.
6. Evaluate the result using practical metrics such as accuracy, classification report, MAE, RMSE, or R2 score depending on the target type.
7. Write observations in a short and direct way so the project can be explained easily.

How to Run

1. Open the notebook in Jupyter Notebook, JupyterLab, Google Colab, or VS Code.
2. Keep the dataset files in the same folder as the notebook.
3. Run the cells from top to bottom.
4. Read the printed metrics and notes at the end before making conclusions.

Final Note

This version keeps the logic understandable and avoids unnecessary complexity. The aim is not to make the notebook look overloaded, but to make it readable, explainable, and useful for a student-level data science portfolio.

Executed Notebook Outputs

The outputs below were generated from the saved notebook after running the code cells in this project folder.

Cell 2 - 2. Load the dataset

```
Dataset shape: (10999, 12)

ID Warehouse_block Mode_of_Shipment Customer_care_calls Customer_rating \
0 1 D Flight 3 1
1 2 F Flight 4 5
2 3 A Flight 2 1
3 4 B Flight 4 4
4 5 C Flight 2 3

Cost_of_the_Product Prior_purchases Product_importance Gender \
0 176 4 low F
1 215 3 low M
2 184 3 low M
3 177 5 medium M
4 183 3 medium F

Discount_offered Weight_in_gms Reached.on.Time_Y.N
0 43 1253 1
1 58 3106 1
2 49 3370 1
3 9 1168 1
4 46 2508 1
```

Cell 3 - 3. Understand the data

```
Rows: 10999
Columns: 12

column dtype missing_values unique_values
0 ID int64 0 10999
1 Warehouse_block object 0 5
2 Mode_of_Shipment object 0 3
3 Customer_care_calls int64 0 8
4 Customer_rating int64 0 7
5 Cost_of_the_Product int64 0 217
6 Prior_purchases int64 0 11
7 Product_importance object 0 3
8 Gender object 0 2
9 Discount_offered int64 0 67
10 Weight_in_gms int64 0 4081
11 Reached.on.Time_Y.N int64 0 2
```

Cell 4 - 4. Clean the data

```
Shape after simple cleaning: (10999, 12)

ID Warehouse_block Mode_of_Shipment Customer_care_calls Customer_rating \
0 1 D Flight 3 1
1 2 F Flight 4 5
2 3 A Flight 2 1
3 4 B Flight 4 4
4 5 C Flight 2 3

Cost_of_the_Product Prior_purchases Product_importance Gender \
0 176 4 low F
1 215 3 low M
2 184 3 low M
3 177 5 medium M
4 183 3 medium F

Discount_offered Weight_in_gms Reached.on.Time_Y.N
0 43 1253 1
1 58 3106 1
2 49 3370 1
3 9 1168 1
4 46 2508 1
```

Cell 5 - 5. Select the target column

```
Selected target column: Reached.on.Time_Y.N

0 1
1 1
2 1
3 1
4 1
Name: Reached.on.Time_Y.N, dtype: int64
```

Cell 6 - 6. Quick EDA

```
Numeric columns: 8
Categorical columns: 4
No missing values found in the previewed columns.

count mean std min 25% 50% \
ID 10999.0 5500.000000 3175.282140 1.0 2750.5 5500.0
Customer_care_calls 10999.0 4.047732 1.410349 1.0 3.0 4.0
Customer_rating 10999.0 2.990272 1.629592 0.0 2.0 3.0
Cost_of_the_Product 10999.0 210.207473 48.072504 95.0 170.0 214.0
```

```
Prior_purchases 10999.0 3.562778 1.730486 1.0 2.0 3.0
Discount_offered 10999.0 13.375034 16.218099 0.0 4.0 7.0
Weight_in_gms 10999.0 3633.877262 1635.353799 982.0 1839.0 4151.0
Reached.on.Time_Y.N 10999.0 0.596691 0.490584 0.0 0.0 1.0
```

```
75% max
ID 8249.5 10999.0
Customer_care_calls 5.0 8.0
Customer_rating 4.0 6.0
Cost_of_the_Product 251.0 311.0
Prior_purchases 4.0 11.0
Discount_offered 11.0 66.0
Weight_in_gms 5048.0 7841.0
Reached.on.Time_Y.N 1.0 1.0
```

Cell 7 - 7. Prepare features and target

```
Dropped ID-like columns: ['ID']
Using 1200 sampled rows for model training speed.
Problem type: classification
Feature shape: (1200, 10)
Target unique values: 2
```

Cell 8 - 8. Train a baseline model

```
Training completed.
```

Cell 9 - 9. Evaluate the model

```
Accuracy: 0.6542
```

```
Classification report:
precision recall f1-score support
```

```
0 0.55 0.66 0.60 95
1 0.75 0.65 0.69 145
```

```
accuracy 0.65 240
macro avg 0.65 0.66 0.65 240
weighted avg 0.67 0.65 0.66 240
```

